

LangGraph Detailed Documentation

Concepts, Practical Patterns, and Simple Python Examples

Prepared for	Selvaraj Dhaamodaran, Principal Software Engineer / Blockchain Architect
Purpose	Enable architects and engineers to understand and apply LangGraph for stateful, controllable AI workflows.
Generated on	15 July 2026
Scope	LangGraph basics, state management, routing, loops, HITL, memory, tool-calling agents, enterprise guidance, and reusable examples.

Executive summary: LangGraph is best used when an AI solution needs explicit workflow control, persistent state, conditional routing, loops, tool use, human review, and long-running execution. This guide uses simple deterministic examples first, then adds LLM and agent patterns so teams can learn incrementally.

Table of Contents

- 1. What is LangGraph?
- 2. When to use LangGraph vs LCEL/chains vs agents
- 3. Core mental model: State, Nodes, Edges, Graph
- 4. Environment setup
- 5. Simple example 1: Hello World graph
- 6. Simple example 2: Content pipeline
- 7. State reducers and accumulated logs
- 8. Conditional routing
- 9. Cycles and safe iteration
- 10. Human-in-the-loop with interrupts
- 11. Tool-calling agent with graph memory
- 12. Enterprise architecture patterns
- 13. Testing, observability, deployment, and governance
- 14. Troubleshooting cheat sheet
- 15. References

1. What is LangGraph?

LangGraph is a low-level orchestration framework for building long-running, stateful AI workflows and agents. It models an application as a graph where functions execute as nodes, edges determine control flow, and a shared state object carries data between steps.

Unlike a simple prompt chain, a graph can branch, loop, pause for human approval, persist state, resume after interruption, and coordinate tool execution. You can use LangGraph with LangChain components, but the core graph orchestration model is useful even when your nodes are plain Python functions.

Practical interpretation: Use LangGraph when you need deterministic control around an LLM: enforce review steps, repeat until quality criteria are met, persist memory, call tools safely, and produce an auditable workflow trace.

2. When to use LangGraph vs chains vs agents

Scenario	Use Chain / LCEL	Use Agent	Use LangGraph
Fixed prompt → model → parser flow	Best fit	Usually unnecessary	Only if future branching/state is expected
Tool choice is dynamic but workflow can remain flexible	Limited	Good fit	Good if you need explicit guardrails around tool loop
Multi-step workflow with branch/loop/approval	Poor fit	Too implicit	Best fit
Long-running business process needing restart/resume	Poor fit	Possible but hard to audit	Best fit with checkpointers
Regulated workflow: human sign-off, audit trail, controlled routing	Limited	Risky if fully autonomous	Best fit

3. Core mental model: State, Nodes, Edges, Graph

State

A typed shared data structure, commonly a TypedDict or Pydantic model. Nodes read from state and return partial updates.

Node

A Python function or runnable component that performs one task, for example classify request, call model, validate result, call API, or write audit log.

Edge

A connection that decides what runs after a node. Edges can be fixed or conditional.

Graph builder

StateGraph registers nodes and edges, then compile() returns a runnable graph.

START and END

Special markers that define entry and termination points.

4. Environment setup

Recommended setup for a local Python development environment:

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate
python -m pip install -U langgraph langchain-core
# Optional when using LangChain model providers:
python -m pip install -U langchain-openai langchain-anthropic
```

Security note: Never hardcode API keys. Use environment variables, managed secrets, or enterprise vault services such as Azure Key Vault, AWS Secrets Manager, or HashiCorp Vault.

5. Simple example 1: Hello World graph

This example uses a built-in message state and a mock node, so it runs without an LLM provider key.

```
from langgraph.graph import StateGraph, MessagesState, START, END

def mock_llm(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}

builder = StateGraph(MessagesState)
builder.add_node("mock_llm", mock_llm)
builder.add_edge(START, "mock_llm")
builder.add_edge("mock_llm", END)

graph = builder.compile()
result = graph.invoke({"messages": [{"role": "user", "content": "hi"}]})
print(result["messages"][-1].content if hasattr(result["messages"][-1], "content") else
      result["messages"][-1]["content"])
```

Key learning: StateGraph defines shared state; add_node registers work; add_edge defines execution order; compile creates a runnable graph; invoke executes it with an initial state.

6. Simple example 2: Content publishing pipeline

A deterministic two-node workflow that drafts and formats content. This is intentionally simple so the graph mechanics are clear.

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class PipelineState(TypedDict):
    topic: str
    draft: str
```

```

    final_content: str

def draft_content(state: PipelineState):
    return {"draft": f"Draft article about {state['topic']}. It explains the key ideas."}

def format_content(state: PipelineState):
    formatted = f"# {state['topic']}\n\n{state['draft']}\n\nConclusion: keep it practical."
    return {"final_content": formatted}

builder = StateGraph(PipelineState)
builder.add_node("draft", draft_content)
builder.add_node("format", format_content)
builder.add_edge(START, "draft")
builder.add_edge("draft", "format")
builder.add_edge("format", END)

pipeline = builder.compile()
result = pipeline.invoke({"topic": "LangGraph", "draft": "", "final_content": ""})
print(result["final_content"])

```

7. State reducers and accumulated logs

By default, field updates overwrite previous values. Reducers customize how updates are merged. A common pattern is using `operator.add` to accumulate list entries such as audit logs, routed decisions, validation messages, or tool results.

```

from typing import Annotated
from typing_extensions import TypedDict
import operator
from langgraph.graph import StateGraph, START, END

class PublishingState(TypedDict):
    topic: str
    draft: str
    final_content: str
    edit_log: Annotated[list[str], operator.add]

def draft_node(state: PublishingState):
    return {
        "draft": f"Initial draft about {state['topic']}",
        "edit_log": ["draft_node created first draft"]
    }

def format_node(state: PublishingState):
    return {
        "final_content": state["draft"].upper(),
        "edit_log": ["format_node converted draft to uppercase"]
    }

builder = StateGraph(PublishingState)
builder.add_node("draft", draft_node)
builder.add_node("format", format_node)
builder.add_edge(START, "draft")
builder.add_edge("draft", "format")
builder.add_edge("format", END)

graph = builder.compile()
print(graph.invoke({"topic": "AI governance", "draft": "", "final_content": "", "edit_log": []}))

```

8. Conditional routing

Conditional routing keeps nodes focused. Instead of one large function with many if/else blocks, let LangGraph route state to specialized nodes.

```
from typing_extensions import Literal, TypedDict
from langgraph.graph import StateGraph, START, END

class RouteState(TypedDict):
    content_type: str
    draft: str
    final_content: str

def draft(state: RouteState):
    return {"draft": "LangGraph helps model workflows as graphs."}

def format_blog(state: RouteState):
    return {"final_content": "Blog post:\n" + state["draft"]}

def format_social(state: RouteState):
    return {"final_content": "Social update: " + state["draft"][:120] + " #AI #LangGraph"}

def route_by_type(state: RouteState) -> Literal["format_blog", "format_social"]:
    return "format_social" if state["content_type"] == "social" else "format_blog"

builder = StateGraph(RouteState)
builder.add_node("draft", draft)
builder.add_node("format_blog", format_blog)
builder.add_node("format_social", format_social)
builder.add_edge(START, "draft")
builder.add_conditional_edges("draft", route_by_type)
builder.add_edge("format_blog", END)
builder.add_edge("format_social", END)

graph = builder.compile()
print(graph.invoke({"content_type": "social", "draft": "", "final_content": ""})["final_content"])
```

9. Cycles and safe iteration

Cycles are useful for retry, refinement, validation, evaluator-optimizer, and approval loops. Always include stop conditions and runtime recursion limits to avoid infinite execution.

```
from typing_extensions import Literal, TypedDict
from langgraph.graph import StateGraph, START, END

class ReviewState(TypedDict):
    draft: str
    score: int
    iteration: int

def write(state: ReviewState):
    return {"draft": "short draft", "score": 4, "iteration": 1}

def improve(state: ReviewState):
    new_score = min(state["score"] + 3, 10)
    return {"draft": state["draft"] + " + improved", "score": new_score, "iteration": state["iteration"] + 1}

def should_continue(state: ReviewState) -> Literal["improve", "end"]:
    return "improve" if state["score"] < 8 and state["iteration"] < 3 else "end"
```

```

builder = StateGraph(ReviewState)
builder.add_node("write", write)
builder.add_node("improve", improve)
builder.add_edge(START, "write")
builder.add_conditional_edges("write", should_continue, {"improve": "improve", "end": END})
builder.add_conditional_edges("improve", should_continue, {"improve": "improve", "end": END})

graph = builder.compile()
print(graph.invoke({"draft": "", "score": 0, "iteration": 0}, config={"recursion_limit": 10}))

```

10. Human-in-the-loop with interrupts

Human-in-the-loop lets a graph pause, expose state to a reviewer, and resume with a decision. This is valuable for regulated workflows, publishing approvals, KYC/AML reviews, incident severity confirmation, and high-risk actions.

```

from typing_extensions import Literal, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command

class ApprovalState(TypedDict):
    draft: str
    feedback: str
    status: str

def write_draft(state: ApprovalState):
    return {"draft": "Proposed response for review."}

def review(state: ApprovalState) -> Command[Literal["publish", "revise"]]:
    decision = interrupt({
        "draft": state["draft"],
        "instruction": "Reply with approve or revise: <feedback>"
    })
    if str(decision).lower().startswith("approve"):
        return Command(goto="publish")
    return Command(goto="revise", update={"feedback": str(decision).replace("revise:", "").strip()})

def revise(state: ApprovalState):
    return {"draft": state["draft"] + " Revised using feedback: " + state["feedback"]}

def publish(state: ApprovalState):
    return {"status": "published"}

builder = StateGraph(ApprovalState)
builder.add_node("write", write_draft)
builder.add_node("review", review)
builder.add_node("revise", revise)
builder.add_node("publish", publish)
builder.add_edge(START, "write")
builder.add_edge("write", "review")
builder.add_edge("revise", "review")
builder.add_edge("publish", END)

graph = builder.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "approval-001"}}
# first = graph.invoke({"draft": "", "feedback": "", "status": ""}, config=config)
# revised = graph.invoke(Command(resume="revise: add examples"), config=config)

```

```
# final = graph.invoke(Command(resume="approve"), config=config)
```

11. Tool-calling agent with graph memory

A common agent pattern is assistant → tools → assistant. The assistant decides whether tools are needed. ToolNode executes tool calls, then the assistant observes the result and responds. Compile with a checkpointer to preserve conversation history by thread_id.

```
from typing_extensions import Literal
from langchain_core.tools import tool
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.memory import MemorySaver

@tool
def count_words(text: str) -> str:
    """Count words in text."""
    return f"Word count: {len(text.split())}"

# llm_with_tools = llm.bind_tools([count_words])

def assistant(state: MessagesState):
    # response = llm_with_tools.invoke(state["messages"])
    # return {"messages": [response]}
    return {"messages": [{"role": "ai", "content": "Tool-ready assistant placeholder"}]}

def should_use_tools(state: MessagesState) -> Literal["tools", "end"]:
    last = state["messages"][-1]
    return "tools" if getattr(last, "tool_calls", None) else "end"

builder = StateGraph(MessagesState)
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode([count_words]))
builder.add_edge(START, "assistant")
builder.add_conditional_edges("assistant", should_use_tools, {"tools": "tools", "end": END})
builder.add_edge("tools", "assistant")

agent = builder.compile(checkpointer=MemorySaver())
# agent.invoke({"messages": [{"role": "user", "content": "Count words in this sentence"}]},
#              config={"configurable": {"thread_id": "user-123"}})
```

12. Enterprise architecture patterns

Pattern	Graph design idea
Compliance triage graph	Intake request → classify jurisdiction/risk → retrieve relevant policy/RAG → evaluate obligations → human review for high risk → generate auditable response.
Stablecoin onboarding workflow	Capture onboarding data → KYC/AML checks → sanctions screening tool → policy decision → exception queue → approval → minting or rejection instruction.
Document automation	Parse document → extract fields → validate against business rules → route exceptions → generate summary → notify reviewer.

Incident response copilot	Ingest alert → enrich from systems → rank severity → propose remediation → human approval → create ticket/update runbook.
Research assistant	Plan sections → launch worker nodes in parallel → synthesize sections → evaluate completeness → revise until threshold met.

13. Testing, observability, deployment, and governance

Testing checklist

- Unit test every node as a plain Python function.
- Snapshot-test graph outputs for representative inputs.
- Test routing functions with all possible path values.
- Test cycles for accepted and max-iteration paths.
- Mock tool/API failures and verify fallback routing.

Observability checklist

- Trace each graph run with request ID/thread ID.
- Log state transitions without sensitive payloads.
- Capture node latency, tool latency, retries, and exceptions.
- Record human approvals and reviewer comments for audit needs.

Deployment checklist

- Package nodes as a service with explicit dependency versions.
- Use managed secrets; never store API keys in code.
- Apply rate limits and timeouts to model/tool calls.
- Use durable checkpoints for production, not in-memory storage.
- Protect PII and regulated data through masking, access control, and retention policies.

Governance checklist

- Define autonomy boundaries: what the graph can do automatically vs what requires approval.
- Maintain prompt, policy, and workflow versioning.
- Keep evidence for decisions in audit logs.
- Apply model risk controls for regulated decisions.
- Provide fallback/override paths for business continuity.

14. Troubleshooting cheat sheet

Symptom	Likely cause	Fix
Graph loops forever	Missing stop condition or recursion limit	Add explicit max iterations and pass config={"recursion_limit": n}.
State field disappears	Node returned wrong key or schema mismatch	Return only valid state keys and check TypedDict/annotations.
List field is overwritten	No reducer configured	Use Annotated[list[T], operator.add] or a custom reducer.

Conditional route not found	Routing function returns value that does not match node/path map	Use Literal return types and explicit path maps.
Resume fails after interrupt	No checkpointer or inconsistent thread_id	Compile with a checkpointer and reuse the same config configurable.thread_id.
Tool call never executes	LLM not bound to tools or route check is wrong	Use bind_tools(), inspect last message tool_calls, and wire ToolNode.

15. References

- [LangGraph overview - Docs by LangChain](#)
- [Workflows and agents - Docs by LangChain](#)
- [LangGraph examples repository - GitHub](#)
- [LangGraph Tutorial: Build Stateful AI Agents in Python - Real Python](#)
- Internal reference: LangGraph.docx authored by Selvaraj Dhaamodaran — Internal Microsoft 365 document retrieved via enterprise search